

LAB 10 | المختبر 10

Load Forecasting with a Transformer

التنبؤ بالحمل الكهربائي باستخدام Transformer

Self-Attention, Positional Embeddings, Sequence Learning, and MATLAB

الانتباه الذاتي والترميز الموضعي وتعلم المتتاليات باستخدام MATLAB

Prepared by | إعداد: Dr.-Ing. Aouss Gabash



MATLAB Lab 10

المختبر 10

75 min - Intermediate | متوسط



Lecture 10

دروس المحاضرة 10

Read Lesson | اقرأ المحاضرة

→



Download Lab PDF

تحميل المختبر بصيغة PDF

Prepared by Dr.-Ing. Aouss Gabash

→

Laboratory Objectives

- Prepare hourly load sequences for transformer regression.

- Create feature and positional embeddings.
- Use multi-head self-attention in MATLAB.
- Train a one-step-ahead forecasting model.
- Compare Transformer and persistence results.
- Analyze forecasting errors.

أهداف المختبر

- تجهيز متتاليات حمل ساعية للانحدار.
- إنشاء تمثيل للميزات والمواقع.
- استخدام الانتباه الذاتي متعدد الرؤوس.
- تدريب نموذج لتوقع الساعة التالية.
- مقارنة Transformer بخط الأساس.
- تحليل أخطاء التنبؤ.

1. Software Requirement

Use MATLAB R2023b or newer with Deep Learning Toolbox because this lab uses `selfAttentionLayer` and `positionEmbeddingLayer`.

1. متطلبات البرنامج

يتطلب المختبر MATLAB R2023b أو أحدث مع Deep Learning Toolbox.

2. Forecasting Task

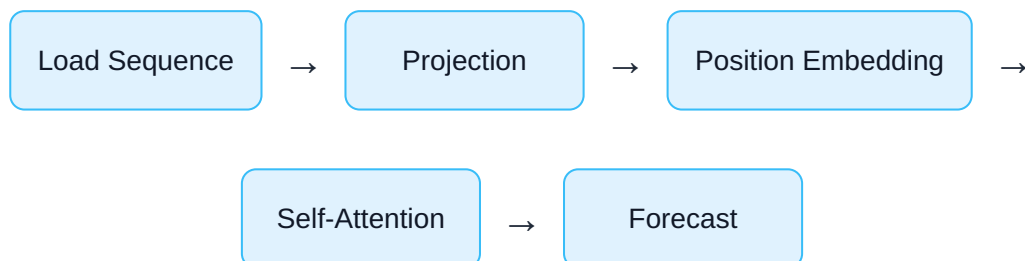
$$[P_{t-47}, \dots, P_t] \rightarrow P_{t+1}$$

Input window: 48 hours. Output: next-hour load.

2. مسألة التنبؤ

نستخدم آخر 48 ساعة لتوقع حمل الساعة التالية، بحيث يرى النموذج نمط يوميين متتاليين.

3. Transformer Workflow



3. تسلسل عمل Transformer

يحوّل النموذج المتتالية إلى تمثيل عددي، ويضيف معلومات الموضع، ثم يستخدم الانتباه الذاتي قبل إنتاج التنبؤ.

4. Complete MATLAB Script

```

%% Lab 10: Transformer Load Forecasting
% MATLAB R2023b or newer
clear; clc; close all;
rng(10);

%% Generate hourly load data
nDays = 240;
N = 24*nDays;
t = (0:N-1)';
hour = mod(t,24);
day = floor(t/24);
dow = mod(day,7);

temperature = 17 ...
    + 9*sin(2*pi*(hour-14)/24) ...
    + 6*sin(2*pi*day/240) ...
    + 1.5*randn(N,1);

daily = 115 ...
    + 24*sin(2*pi*(hour-7)/24) ...
    + 9*sin(4*pi*(hour-5)/24);
weekly = 9*(dow<5) - 8*(dow>=5);
seasonal = 10*sin(2*pi*day/240);

loadMW = daily + weekly + seasonal ...
    + 0.8*max(temperature-23,0) ...
    + 0.45*max(11-temperature,0) ...
    + 2.2*randn(N,1);

%% Create 48-hour sequence-to-one samples
seqLen = 48;
numSamples = N-seqLen;
X = cell(numSamples,1);
Y = zeros(numSamples,1);
for k = 1:numSamples
    X{k} = loadMW(k:k+seqLen-1)';
    Y(k) = loadMW(k+seqLen);
end

%% Chronological split
nTrain = floor(0.70*numSamples);
nVal = floor(0.15*numSamples);
iTrain = 1:nTrain;
iVal = nTrain+1:nTrain+nVal;
iTest = nTrain+nVal+1:numSamples;

```

```

XTrain=X(iTrain); YTrain=Y(iTrain);
XVal=X(iVal); YVal=Y(iVal);
XTest=X(iTest); YTest=Y(iTest);

%% Normalize using training data only
trainMatrix = cell2mat(XTrain);
mu = mean(trainMatrix,'all');
sigma = std(trainMatrix,0,'all');
XTrainN = cellfun(@(x)(x-mu)/sigma,XTrain,'UniformOutput',false);
XValN = cellfun(@(x)(x-mu)/sigma,XVal,'UniformOutput',false);
XTestN = cellfun(@(x)(x-mu)/sigma,XTest,'UniformOutput',false);
YTrainN = (YTrain-mu)/sigma;
YValN = (YVal-mu)/sigma;

%% Transformer architecture
numFeatures = 1;
embedDim = 32;
numHeads = 4;
ffnDim = 64;

layers = [
    sequenceInputLayer(numFeatures,Name="input")
    fullyConnectedLayer(embedDim,Name="projection")
    positionEmbeddingLayer(embedDim,seqLen,Name="position")
    additionLayer(2,Name="add_position")
    selfAttentionLayer(numHeads,embedDim, ...
        DropoutProbability=0.10,Name="attention")
    additionLayer(2,Name="add_attention")
    layerNormalizationLayer(Name="norm1")
    fullyConnectedLayer(ffnDim,Name="ffn1")
    reluLayer(Name="relu")
    dropoutLayer(0.10,Name="dropout")
    fullyConnectedLayer(embedDim,Name="ffn2")
    additionLayer(2,Name="add_ffn")
    layerNormalizationLayer(Name="norm2")
    globalAveragePooling1dLayer(Name="pool")
    fullyConnectedLayer(1,Name="forecast")
    regressionLayer(Name="output")
];

lgraph = layerGraph(layers);
lgraph = connectLayers(lgraph,"projection","add_position/in2");
lgraph = connectLayers(lgraph,"add_position","add_attention/in2");
lgraph = connectLayers(lgraph,"norm1","add_ffn/in2");

```

```

%% Training
options = trainingOptions("adam", ...
    MaxEpochs=70, ...
    MiniBatchSize=64, ...
    InitialLearnRate=1e-3, ...
    GradientThreshold=1, ...
    Shuffle="never", ...
    ValidationData={XValN,YValN}, ...
    ValidationFrequency=30, ...
    ValidationPatience=8, ...
    Verbose=false, ...
    Plots="training-progress");

net = trainNetwork(XTrainN,YTrainN,lgraph,options);

%% Prediction and inverse normalization
YPredN = predict(net,XTestN,MiniBatchSize=64);
YPred = YPredN*sigma + mu;
YPersistence = cellfun(@(x)x(end),XTest);

%% Performance metrics
metric = @(yp,yt) [ ...
    mean(abs(yp-yt)), ...
    sqrt(mean((yp-yt).^2)), ...
    100*mean(abs((yp-yt)./yt)) ];

mBase = metric(YPersistence,YTest);
mTrans = metric(YPred,YTest);
Results = array2table([mBase;mTrans], ...
    VariableNames={"MAE_MW","RMSE_MW","MAPE_percent"}, ...
    RowNames={"Persistence","Transformer"});
disp(Results)

%% Forecast plots
nShow = min(240,numel(YTest));
figure
plot(YTest(1:nShow),"k",LineWidth=1.35); hold on
plot(YPersistence(1:nShow),"--",LineWidth=1.0)
plot(YPred(1:nShow),LineWidth=1.2)
grid on
xlabel("Test Hour"); ylabel("Load (MW)")
legend("Actual","Persistence","Transformer",Location="best")
title("Transformer One-Step-Ahead Load Forecast")

```



```
%% Error analysis
e = YPred-YTest;
figure
histogram(e,30)
grid on
xlabel("Prediction Error (MW)"); ylabel("Frequency")
title("Transformer Error Distribution")

figure
scatter(YTest,YPred,10,"filled")
grid on; axis equal
xlabel("Actual Load (MW)")
ylabel("Predicted Load (MW)")
title("Actual versus Predicted Load")
```

4. البرنامج الكامل في MATLAB

ينشئ البرنامج بيانات حمل، ويبنّي نوافذ بطول 48 ساعة، ويطبّعها باستخدام بيانات التدريب فقط، ثم يدرب Transformer صغيرًا ويقارنه بخط أساس.

5. Core Components

Projection Layer

Maps the input feature to the embedding dimension.

Position Embedding

Adds temporal-order information.

Self-Attention

Lets each time step attend to other steps.

Residual Connections

Support stable gradient flow.

5. المكونات الأساسية

يتكون النموذج من إسقاط للميزات وترميز موضعي وانتباه ذاتي ووصلات متبقية وطبقات تغذية أمامية.

6. Evaluation Metrics

$$MAE = (1/N) \sum |\hat{y}_i - y_i|$$

Missing argument for sqrt

$$MAPE = (100/N) \sum |(\hat{y}_i - y_i)/y_i|$$

6. مؤشرات التقييم

تستخدم MAE و RMSE و MAPE لتقييم دقة التنبؤ ومقارنتها بخط الأساس.

7. Experiments

1. Change the input window to 24, 72, or 168 hours.
2. Change the embedding dimension.
3. Change the number of attention heads.
4. Add temperature as a second feature.
5. Compare Transformer and LSTM on the same split.

7. التجارب

1. غير طول نافذة الإدخال.
2. غير بُعد التمثيل.
3. غير عدد رؤوس الانتباه.

4. أضف درجة الحرارة كميزة ثانية.

5. قارن Transformer وLSTM على التقسيم نفسه.

8. Lab Tasks

1. Run the complete script.
2. Record Transformer and persistence metrics.
3. Plot predictions and error distribution.
4. Explain the role of positional embeddings.
5. Explain the residual connections.
6. Repeat at least two experiments.

8. مهام المختبر

1. شغل البرنامج الكامل.
2. سجل مؤشرات Transformer وخط الأساس.
3. ارسم التنبؤات وتوزيع الأخطاء.
4. اشرح دور الترميز الموضعي.
5. اشرح الوصلات المتبقية.
6. نفذ تجربتين على الأقل.



Review Questions

1. Why does a Transformer need positional information?
2. What does self-attention calculate?
3. Why use multiple attention heads?
4. Why is chronological splitting required?
5. Why compare with persistence?
6. How can weather inputs improve forecasting?

أسئلة المراجعة

1. لماذا يحتاج Transformer إلى معلومات الموضع؟
2. ماذا يحسب الانتباه الذاتي؟
3. لماذا نستخدم عدة رؤوس انتباه؟
4. لماذا نحتاج إلى تقسيم زمني؟

5. لماذا نقارن بخط أساس؟

6. كيف تحسن بيانات الطقس التنبؤ؟

References | المراجع

المراجع العامة المستخدمة في هذا المقرر

[1] A. Gabash, *Flexible Optimal Operations of Energy Supply Networks with Renewable Energy Generation and Battery Storage*. Saarbrücken, Germany: Südwestdeutscher Verlag für Hochschulschriften, 2014.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Global ed. Harlow, U.K.: Pearson Education Limited, 2022.

[3] A. Gabash, "Energy Market Transition and Climate Change: A Review of TSOs-DSOs C++ Framework from 1800 to Present," *Energies*, vol. 16, no. 17, Art. no. 6139, 2023, doi: 10.3390/en16176139.

[4] A. Gabash, *AI Applications in Electrical Power Systems*, Version 1.0, 2026. [Online]. Available: <https://aoussgabash.com>

Publication Information | معلومات النشر

Document	Laboratory 10 · Load Forecasting with a Transformer
Course	AI Applications in Electrical Power Systems
Author	Dr.-Ing. Aouss Gabash
Version	1.0
Publication year	2026
Official website	https://aoussgabash.com
Citation style	IEEE

Recommended citation:

Gabash, A. (2026). Load Forecasting with a Transformer. AI Applications in Electrical Power Systems, Laboratory 10, Version 1.0. <https://aoussgabash.com>

المرجع المقترح:

أوس غباش، 2026، مخبر 10، تطبيقات الذكاء الاصطناعي في أنظمة الطاقة الكهربائية، الإصدار 1.0، <https://aoussgabash.com>

© 2026 Dr.-Ing. Aouss Gabash. Educational use with attribution to the author and official website.